

Atención con puntero para el WTAP

Un encoder con self-attention y un puntero que aprenden a asignar armas a objetivos imitando paso a paso a un experto (ILS)

[Nombre(s) del grupo]

Optimización combinatoria · 2026

La decisión puntual que toma el modelo

Dado el estado actual (qué armas ya se asignaron y a qué objetivos), **¿qué arma asigno ahora y a qué objetivo?**

W armas, T objetivos. El objetivo j vale V_j ; el arma i lo destruye con prob. p_{ij} . Cada arma va a ≤ 1 objetivo; un objetivo puede recibir varias.

$$\min_x Z = \sum_{j=1}^T V_j \prod_{i=1}^W (1 - p_{ij})^{x_{ij}}$$

valor esperado que sobrevive (se minimiza)

Tarea 4 (MLP) → Tarea 5 (atención + puntero)

Pesos por posición

→ pesos compartidos: cada par se evalúa con la misma función

Solo $W = 10$, $T = 8$

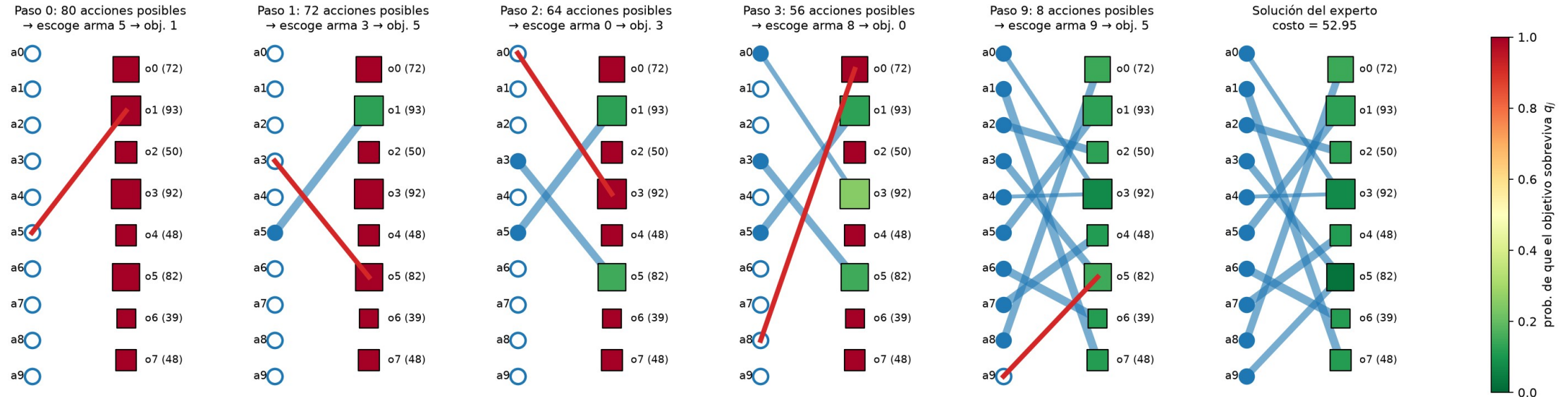
→ ningún peso depende de W ni de T (probado hasta 20×15)

Orden fijo de armas

→ la acción es un par (arma, objetivo): el modelo elige también el arma

De la solución del experto a decisiones

Tuplas (estado, acciones posibles → acción escogida); cada paso agrega un par (arma libre → objetivo). Ejemplo real 10×8



Estado inicial

asignación vacía: las 10 armas libres (análogo a `visited=[0]`)

Acciones posibles

todos los pares (arma libre, objetivo): $k \cdot T$
80 en el paso 0 ... 8 en el último

Acción escogida

par del experto con mayor ganancia $V_j \cdot q_j \cdot p_{ij}$ (criterio MMR) → `env.state_transition`

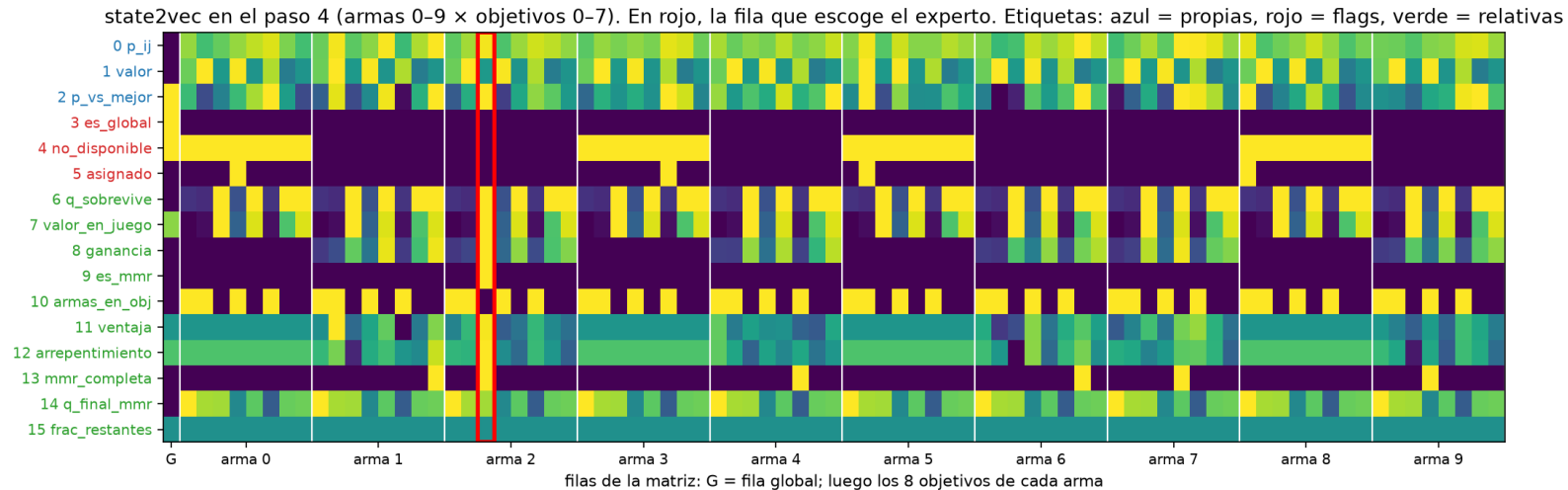
Término

cuando las W armas están asignadas: $W = 10$ tuplas por instancia

¿Por qué ese orden? El experto entrega un conjunto de pares sin orden; ordenarlos por ganancia marginal da una etiqueta única por estado, reconstruye exactamente su solución y es coherente con MMR. En este ejemplo el par del experto difiere del que elegiría MMR en 4 de 10 pasos: eso es lo que el modelo debe aprender a corregir.

Entrada: matriz $(1 + W \cdot T) \times 16$ — una fila por acción posible

81 × 16 en entrenamiento. Todas las filas se procesan con los mismos pesos; ninguna columna depende del índice



Filas

fila 0

global [CLS]: forma la query; siempre enmascarada

$1+i \cdot T+j$

par (arma $i \rightarrow$ objetivo j): estado + acción “asignar i a j ”

Con k armas libres quedan $k \cdot T$ filas válidas = las acciones de `env.gen_actions`.

Propias del par (0-2)

- 0 p_{ij} → efectividad del arma
- 1 $V_j / \max V$ → importancia del objetivo
- 2 $p_{ij} - \max_{j'} p_{ij'}$ → ¿es su mejor destino?

Flags de estado (3-5)

- 3 **es_global** → fila de la query
- 4 **no_disponible** → MÁSCARA (arma ya asignada)
- 5 **asignado** → $x_{ij} = 1$ en la solución parcial

Relativas al estado (6-15)

de las heurísticas: **ganancia** $V_j q_j p_{ij}$ (MMR), **es_mmr**, q_j , **armas en j**, **ventaja** y **arrepentimiento** vs otras armas libres, **completar con MMR** (anticipación), **fracción restante**

Independencia del tamaño: todo está normalizado por instancia ($\div$ máximo o fracciones) y la información global se repite en cada fila y en la fila 0. La self-attention deja que el par (i, j) compare con los pares (i', j) de otras armas.

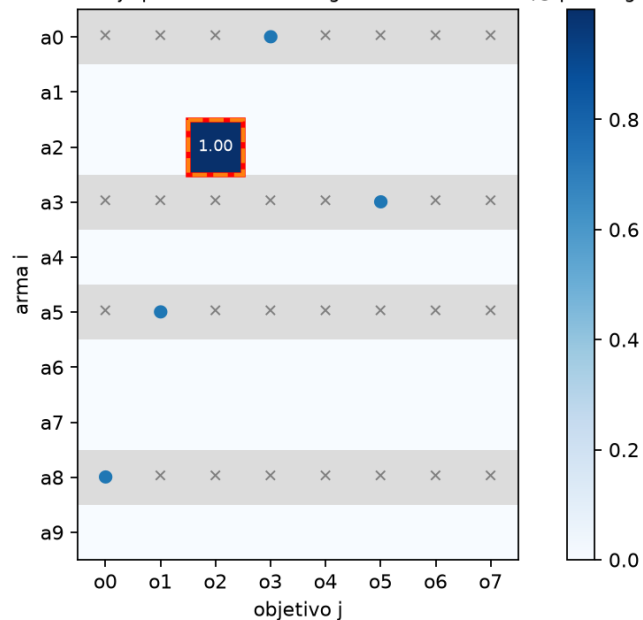
Salida: un logit por fila, máscara y etiqueta

Modelo: estructura de TSP_Attention; cambian VEC_LEN, la query de decisión (paso 3) y la columna de máscara (paso 4)



tamaño moderado: $d = 64 \cdot 2$ capas · 75.776 parámetros · Dropout evaluado en 0 / 0,1 / 0,2 (elegido por gap de validación: 0,0)

Salida del puntero en el paso 4 (probabilidad de cada par)
rojo = experto · naranja puntuado = MMR · gris = enmascarado (● par asignado)



SALIDA Y ETIQUETA

Logits: vector de largo $1 + W \cdot T$ (uno por fila).
Acción ↔ fila: ("constructive", (i, j)) = fila $1 + i \cdot T + j$.
Máscara: fila global y pares de armas ya asignadas ($\text{no_disponible} = 1$) → prob. 0.
Y: índice entero de la fila del experto ($\equiv$ one-hot).
Pérdida: $\text{CrossEntropyLoss}(\text{model}(x), Y)$.

	TSP_Attention	WTAP_Attention
VEC_LEN	6	16
Query (paso 3)	$[h_{\text{actual}} ; h_{\text{inicial}}]$ col. 2 + fila 0	$[h_{\text{global}} ; \text{promedio disp.}]$ col. 3 (es_global)
Máscara (paso 4)	visitada (col. 3)	no_disponible (col. 4)
Encoder	1 bloque	2 bloques + LayerNorm + Dropout

Resultados: entrenamiento y evaluación (pasos opcionales 7 y 8)

60.000 pares (estado, fila) de 6.000 instancias · prueba en las mismas 50 instancias de la Tarea 4 · mismo GreedyAgent

87,3%

accuracy validación

decisiones iguales al experto

2,97%

gap modelo

óptimo en 30% de las instancias

4,42%

gap MLP (Tarea 4)

orden fijo de armas

7,68%

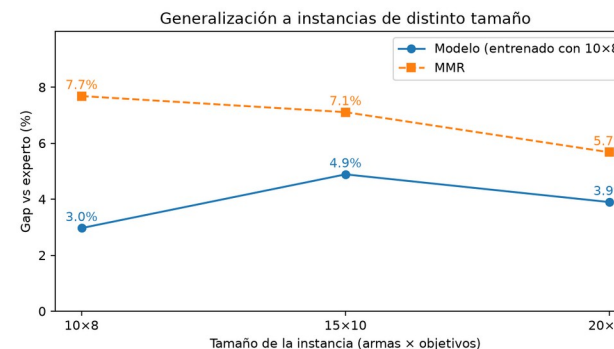
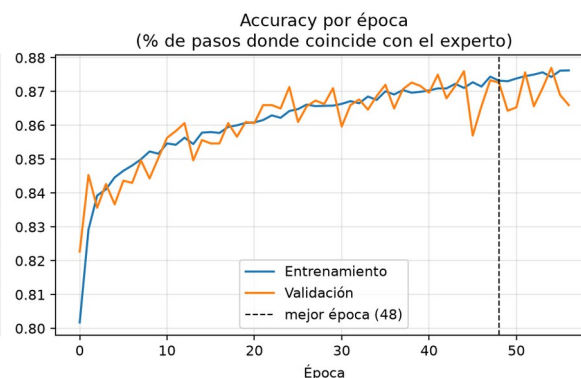
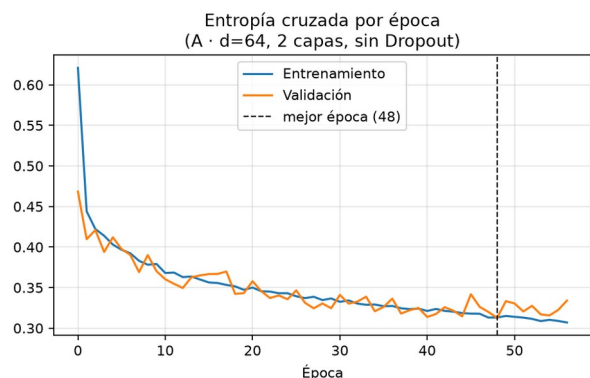
gap MMR

heurística clásica

72%

gana a MMR

empata en 20%



GENERALIZA

Entrenado solo con 10×8, se usa sin cambios en 15×10 y 20×15 (151 y 301 filas).

Lectura. Con la misma receta de imitación, el puntero sobre pares mejora a la MLP de la Tarea 4 (2,97% vs 4,42%) y reduce el gap de MMR (7,68%), pero ahora decide también qué arma usar, con pesos compartidos y para cualquier tamaño.

Gap vs experto por tamaño: 10×8: 3,0% (MMR 7,7%) · 15×10: 4,9% (MMR 7,1%) · 20×15: 3,9% (MMR 5,7%).

Dropout: gap de validación 0,0 → 3,35% · 0,1 → 3,51% · 0,2 → 3,90%. Sin Dropout fue lo mejor: la pérdida de entrenamiento ≈ validación, el modelo no llega a sobreajustar.

Próximo paso: el correo lo deja para después: RL o DAgger para corregir el distribution shift y más capas/cabezas.